

UEC642

DEEP LEARNING AND APPLICATIONS

American Sign Language Detection

**Submitted to:
Dr. Gaganpreet Kaur**



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Aug-Dec 2025

Submitted By:

Ishmit Singh-102215056

Adamyia Sharma-102215132

Sumedha Khosla-102395002

Introduction:

American Sign Language (ASL) is one of the most widely used communication systems among individuals with hearing or speech impairments. Traditional ASL interpretation requires human experts, which limits accessibility and makes real-time communication challenging.

With recent advancements in artificial intelligence and computer vision, technology now offers new opportunities to support more accessible communication. Modern systems are increasingly capable of understanding visual cues, recognising patterns, and interpreting human gestures with greater accuracy and speed.

This project explores the development of an intelligent system that can identify hand gestures associated with the ASL alphabet. By combining deep learning techniques with real-time visual processing, the system aims to provide a flexible foundation for gesture-based communication tools. Such solutions have the potential to enhance accessibility, support inclusive interaction, and serve as a stepping stone toward more advanced sign-language recognition applications in the future.

Dataset:

For this project, we used the ASL Alphabet Dataset from Kaggle: <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>

The dataset contains images representing hand signs used in the American Sign Language (ASL) alphabet. Each image corresponds to one gesture and belongs to one of the ASL alphabet classes. The dataset is designed to support sign-language recognition systems and includes a diverse set of hand poses under different lighting and backgrounds.

The target class corresponds to the ASL letter being shown in the image, making this a multi-class classification task.

Target variable:

- label: Indicates which ASL letter (A–Z) or special class (SPACE, NOTHING, DELETE) the image represents. Thus, the problem contains 29 distinct classes.

Dataset Size:

- Total classes: 29
- Images per class: ~3,000
- Total images: ~87,000
- Original Columns: 1 (Image)
- Problem Type: Multi-class Classification

LITERATURE SURVEY:

Looking at previous work helped us decide the practical trade-offs for the product. We favoured lightweight, real-time-friendly tools like MediaPipe for landmark extraction, examined CNN/RNN/Transformer approaches, and chose a model family that supports continuous, sentence-level recognition rather than only isolated signs. The literature also made clear that public ASL datasets are limited, which motivated us to build a focused ASL alphabet/digits dataset.

The development of real-time communication systems for deaf-mute and hearing individuals has gained significant attention in recent years. Various research efforts have focused on improving sign language recognition, speech-to-sign translation, and bidirectional communication between users. This literature review examines key studies on sign language recognition, machine learning techniques, and speech-based captioning for sign conversion. By analysing existing methodologies, datasets, and research gaps, we aim to identify areas for further enhancement and innovation in this domain.

Table 1 presents various state-of-the-art technologies for sign-to-text recognition, highlighting different approaches to converting sign language gestures into text. Techniques include k-NN for static gestures, HMMs for dynamic gestures, and CNN-based models such as CNN-DSC. MediaPipe is widely used for feature extraction, while LSTM aids in sequence classification. Despite advancements, challenges remain, such as sensitivity to lighting, lack of support for two-handed gestures, and limited standardised ASL datasets. Addressing these gaps can enhance system accuracy and usability.

Table 2 summarises various case studies evaluating sign language recognition and conversion techniques. Research explores deep learning architectures like CNNs, RNNs, and hybrid models, emphasising dataset diversity and benchmarking CSLR techniques. Key challenges include real-time processing, dataset limitations, and the need for more generalised models. Addressing these gaps can improve the inclusivity and efficiency of sign language translation systems.

Table 1. Comparison of various Sign-to-Text Recognition Techniques

Title	Method Used	Research Gap	Dataset	Results
Real-time ASL recognition (2018).	k-NN for static gestures, HMM for dynamic gestures	Sensitivity to lighting, no support for two-handed gestures	24,624 images of ASL digits, letters, and gestures, captured using a smartphone camera.	High accuracy (99.7% static, 97.23% dynamic)
Recognition of Indian Sign Language using Deep Learning Models (2022).	CNN with Depth-wise Separable Convolution (CNN-DSC)	Lack of a standardised ASL dataset	Dataset, including a self-collected dataset with 65 users and a publicly available dataset.	Efficient two-handed gesture recognition, 92.43% accuracy
Automatic Indian Sign Language Recognition using MediaPipe Holistic and LSTM Network (2023).	MediaPipe Holistic for feature extraction, LSTM for classification	Inter-class similarities, dataset inconsistencies	Include the dataset.	Achieves 94.8% accuracy, extracts 3D skeletal key points
Understanding vision-based continuous sign language recognition (2020).	Analysis of CSLR models (HMMs, DTW, CNNs, LSTMs, Transformers)	Real-time processing and occlusion handling remain challenges	RWTH- Phoenix- weather, SIGNUM, Synthetic Vocab.	Focus on robust feature extraction and temporal modelling
Sign Language Recognition and Translation Systems for Enhanced Communication for the Hearing Impaired (2024).	Integration of SLR with ASR, CNNs, LSTMs, and Transformers	Lacks bidirectional communication	Robita ASL and Own dataset created by taking webcam images.	Enhances accuracy, enables gesture-based communication
Real-time SIGN Recognition System to aid deaf-dumb people (2011).	Machine Learning (Simple Threshold Values)	Limited Gesture Set	320 images.	Achieved 98.125% accuracy in real-time recognition when tested on 160 images.

Automatic ASL recognition system (2013).	Image processing, HU-invariant moments, SVM	Limited to controlled environments, no dynamic gestures.	The dataset contains 720 images created in-house.	96.23% accuracy for static gestures
Sign Language Translation Across Multiple Languages (2024).	SVM, CNN, 3D-CNN	No detailed implementation provided	Dataset of 35,000 images.	High accuracy for colour (99.72%) and grayscale (99.90%) images

Table 2. Case Studies for Sign Language Recognition

Title	Method Used	Research Gap	Dataset	Result
Sign Language Recognition: A Deep Survey (2021).	Survey on CNNs, RNNs, hybrid models	Dataset diversity, model robustness, and continuous sign recognition	Multi-Lingual datasets used	Comprehensive analysis of deep learning models
Reviewing 25 years of continuous sign language recognition research: Advances, challenges, and prospects. (2024)	Review of CSLR techniques (HMMs, CNNs, RNNs)	No new methods proposed	Multi-Lingual datasets used	Highlights the importance of large datasets

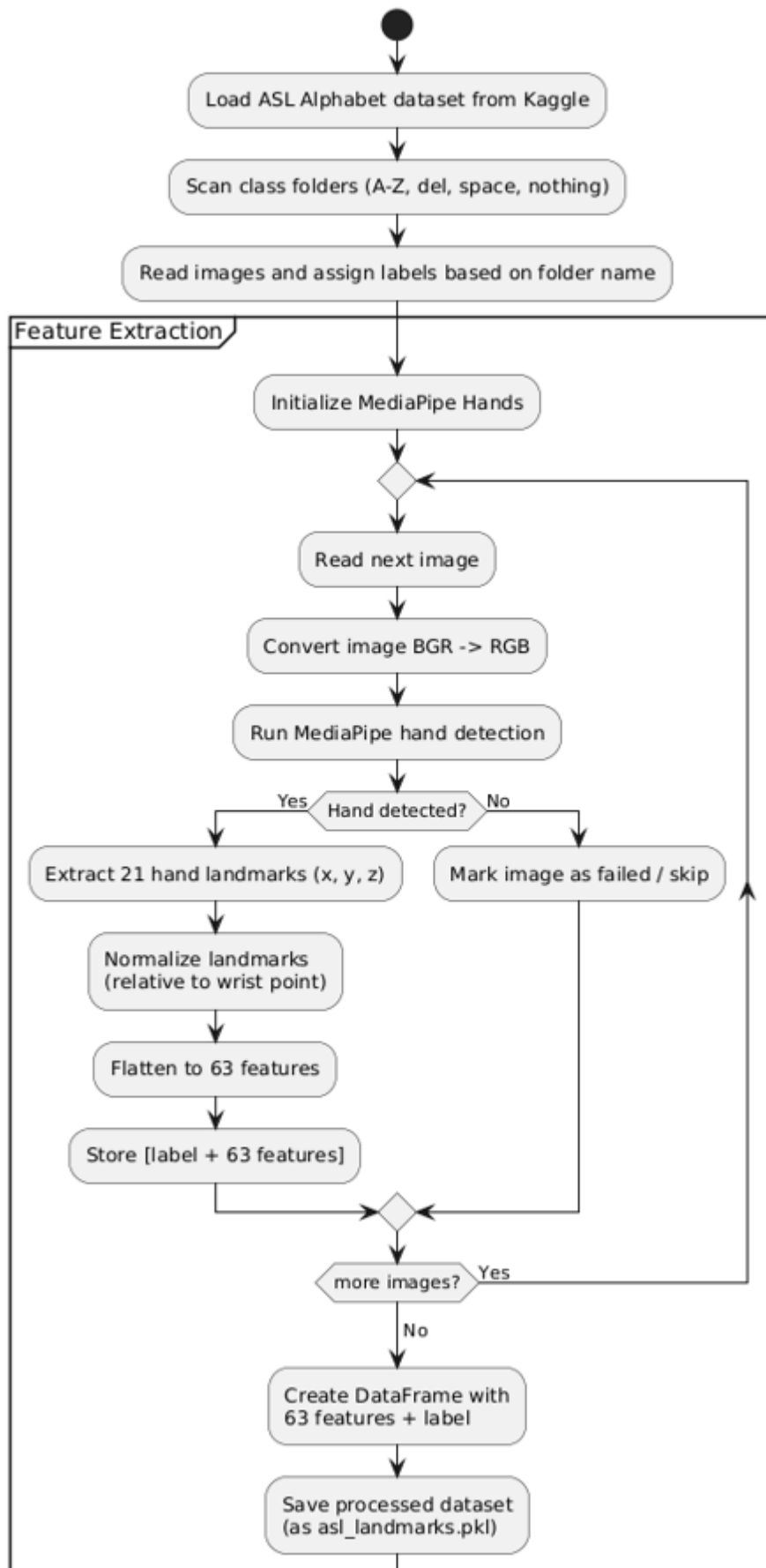
RESEARCH GAPS

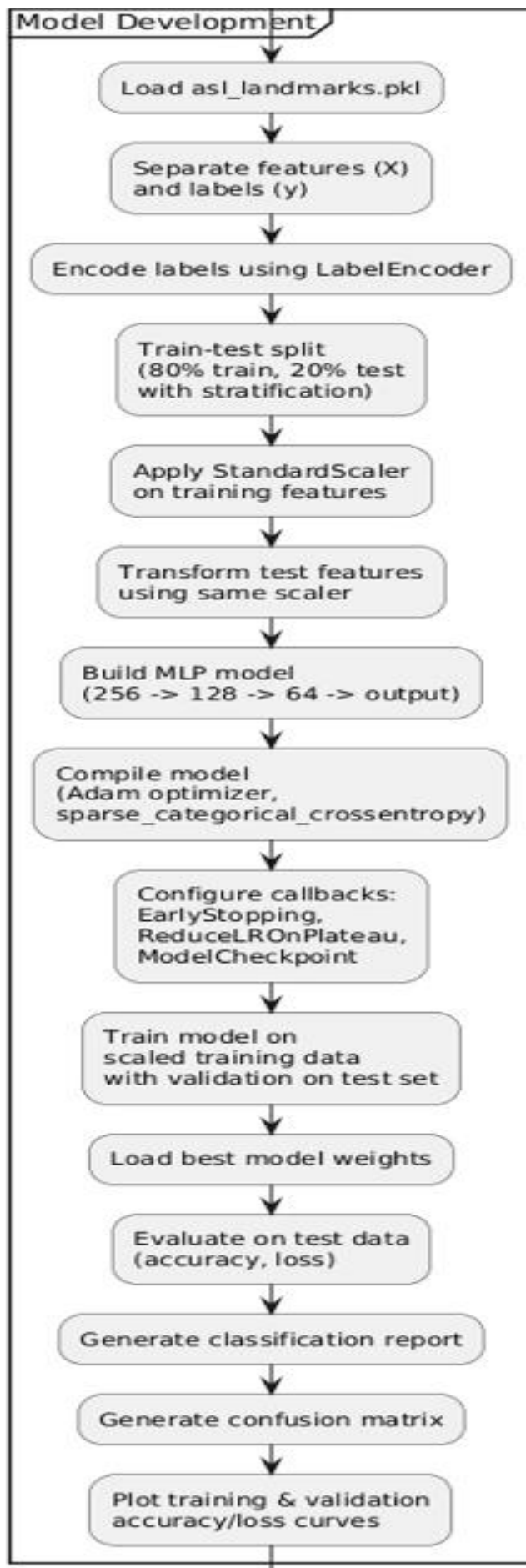
- **Dependence on Human Interpreters:** Expensive and not always available during spontaneous or emergency interactions.
- **Low Accuracy for Complex Signing:** Difficulty recognising dynamic or fast signing styles.
- **Standalone Gesture Apps:** Existing apps only handle basic gestures; they struggle with real-time, fluid signing.
- **One-Way Translation Tools:** Tools like speech-to-text only offer one-directional communication, failing to support inclusive interaction.
- **Wearable-Based Solutions:** Gloves and sensor-based systems are costly, bulky, and impractical for daily use.
- **Latency and Responsiveness Issues:** Many tools show delay and poor real-time response, impacting usability in critical contexts like education or healthcare.
- **Limited Support for Continuous Signing:** Most systems can't handle ongoing sign language streams or multilingual gestures effectively.
- **High Infrastructure Cost:** Existing solutions often require expensive hardware or professionally trained staff, reducing accessibility.
- **Lack of Unified, Open-Source Systems:** Existing solutions handling sign are commercially licensed or complex to use, and lack real-time, offline, multi-input support accessible to the deaf and speech-impaired.

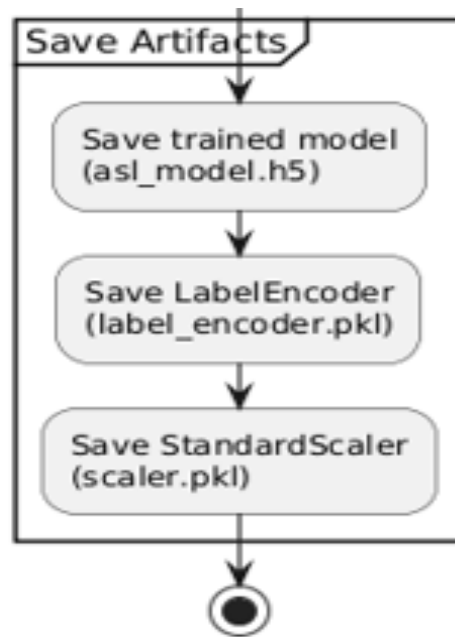
Methodology:

This project follows a structured pipeline to build an ASL alphabet recognition system capable of performing real-time gesture classification. The process involves dataset preparation, feature extraction using hand landmarks, model training using deep learning (MLP classifier), and finally, real-time prediction using webcam input. The major steps are described below.

ASL Alphabet Recognition - Methodology Flow (Offline Pipeline)







1. Dataset Loading

The ASL Alphabet dataset was imported from Kaggle, containing image folders for 29 classes (A–Z, SPACE, DELETE, NOTHING). Using Python, the dataset directory was scanned, and all image files were collected class-wise. Each folder name served as the class label.

Initial inspection included:

- Counting images per class
- Checking directory structure
- Verifying valid image formats (.jpg, .png, .jpeg)

2. Hand Landmark Extraction Using MediaPipe

Before training any model, every image requires preprocessing and structured feature extraction.

a) Using MediaPipe Hands: Google's MediaPipe Hands model was used to detect a single hand and extract 21 key landmarks per image.

b) Converting Images to Landmark-Based Features:

For each image:

- The hand was detected
- 21 landmarks (x, y, z) were extracted
- Wrist point (landmark 0) was used as a reference
- All landmarks were normalised by subtracting wrist coordinates (to reduce variation in hand position and camera distance)

If no hand was detected, the image was marked as failed and excluded.

Each sample was finally represented as:

- 63 numerical features (21 landmarks $\times$ 3 coordinates)

This approach reduces noise and captures the geometric structure of the ASL gesture more efficiently than raw images.

3. Dataset Construction

All extracted landmarks were combined into a Pandas DataFrame with:

- 63 landmark features

- 1 target label (class name)

Column format:

label, x0, y0, z0, x1, y1, z1, ... , x20, y20, z20

The dataset was then saved in pickle format (asl_landmarks.pkl) for model training.

4. **Label Encoding**

Since the target variable (label) consisted of strings (A, B, C, ...), LabelEncoder was used to convert each class into an integer:

- Example: $A \rightarrow 0$, $B \rightarrow 1$, $C \rightarrow 2$, etc.

This numeric encoding is required for training deep learning models.

5. **Train-Test Split**

The dataset was split into:

- 80% training data
- 20% testing data

Stratification ensured that each class had proportional representation in both splits.

6. **Feature Scaling**

Because neural networks perform better when input features are normalised, StandardScaler was applied:

- fit_transform() applied to training data
- transform() applied to test data

This standardised all 63 features to have a mean of 0 and a variance of 1.

7. **Model Building Using MLP Classifier**

The classification model used in this project is a Multilayer Perceptron (MLP) built with TensorFlow/Keras.

Key components included:

- Dense layers: $256 \rightarrow 128 \rightarrow 64$ neurons
- ReLU activation for non-linearity
- Batch Normalization for stable training
- Dropout to reduce overfitting
- Softmax output layer for multi-class classification (29 classes)

The model was compiled using:

- Adam optimizer (learning rate = 0.001)
- sparse_categorical_crossentropy loss
- Accuracy as evaluation metric

Callbacks such as EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint were used to improve performance.

8. **Model Training**

The network was trained for up to 100 epochs with:

- Batch size = 128
- Validation on the test set
- Automatic learning-rate reduction when validation loss plateaued

The best-performing model (highest validation accuracy) was saved.

9. Model Prediction and Evaluation

After training, the best model was evaluated on the test data. Metrics calculated:

- Accuracy (overall correctness)
- Loss (model error)
- Confusion Matrix – showing correct vs incorrect predictions per class
- Classification Report – precision, recall, F1-score for each of the 29 classes

Training and validation curves for accuracy and loss were plotted to assess learning behaviour.

10. Saving the Final Components

Three essential files were saved for real-time inference:

1. asl_model.h5 – trained MLP model
2. scaler.pkl – feature scaler
3. label_encoder.pkl – encoded class mapping

These files were required to deploy the model for live webcam-based recognition.

11. Real-Time Gesture Detection

For real-time prediction:

- Webcam frames were captured using OpenCV
- MediaPipe extracted hand landmarks on each frame
- Landmarks were normalised and scaled using the saved scaler
- The trained model predicted the ASL class
- Prediction was displayed live on the video feed

This completed the end-to-end ASL recognition system using deep learning and computer vision.

Results:

After training the deep learning-based ASL classification model on the extracted hand-landmark features, the model was evaluated on the unseen 20% test split. Multiple performance metrics were calculated to assess how accurately the system can recognise ASL alphabet signs.

• Accuracy:

- Training Accuracy: 99.78%
- Testing Accuracy: 99.53%

This indicates that the model performs extremely well on both training and unseen data. With a test accuracy of 99.53%, the classifier correctly identifies approximately 995 out of every 1000 ASL gestures, demonstrating excellent generalisation.

• Loss Values:

- Training Loss: 0.0064
- Testing Loss: 0.0223

Very low loss values indicate that the neural network has learned the gesture representations

effectively, and no major overfitting is observed due to regularisation techniques such as dropout and batch normalisation.

• **Classification Report:**

A detailed classification report was generated for all 29 classes (A–Z, del, space, nothing), including precision, recall, and F1-score for each class.

	precision	recall	f1-score	support
A	0.995	0.998	0.997	437
B	1.000	1.000	1.000	441
C	0.995	0.997	0.996	398
D	0.998	0.998	0.998	493
E	0.996	0.994	0.995	462
F	0.995	0.998	0.997	575
G	0.998	0.996	0.997	488
H	0.998	1.000	0.999	479
I	0.990	0.990	0.990	477
J	0.994	0.996	0.995	516
K	0.998	1.000	0.999	540
L	1.000	0.996	0.998	505
M	0.966	0.987	0.976	313
N	0.984	0.961	0.972	255
O	0.996	0.991	0.993	453
P	0.998	0.998	0.998	408
Q	0.998	0.998	0.998	419
R	1.000	0.990	0.995	508
S	0.992	1.000	0.996	510

R	1.000	0.990	0.995	508
S	0.992	1.000	0.996	510
T	1.000	0.989	0.995	470
U	0.998	0.996	0.997	503
V	0.996	0.996	0.996	510
W	0.996	0.996	0.996	491
X	0.993	0.993	0.993	432
Y	1.000	1.000	1.000	517
Z	0.996	0.996	0.996	470
del	0.991	1.000	0.996	340
nothing	1.000	1.000	1.000	1
space	0.991	0.997	0.994	325
accuracy			0.995	12736
macro avg	0.995	0.995	0.995	12736
weighted avg	0.995	0.995	0.995	12736

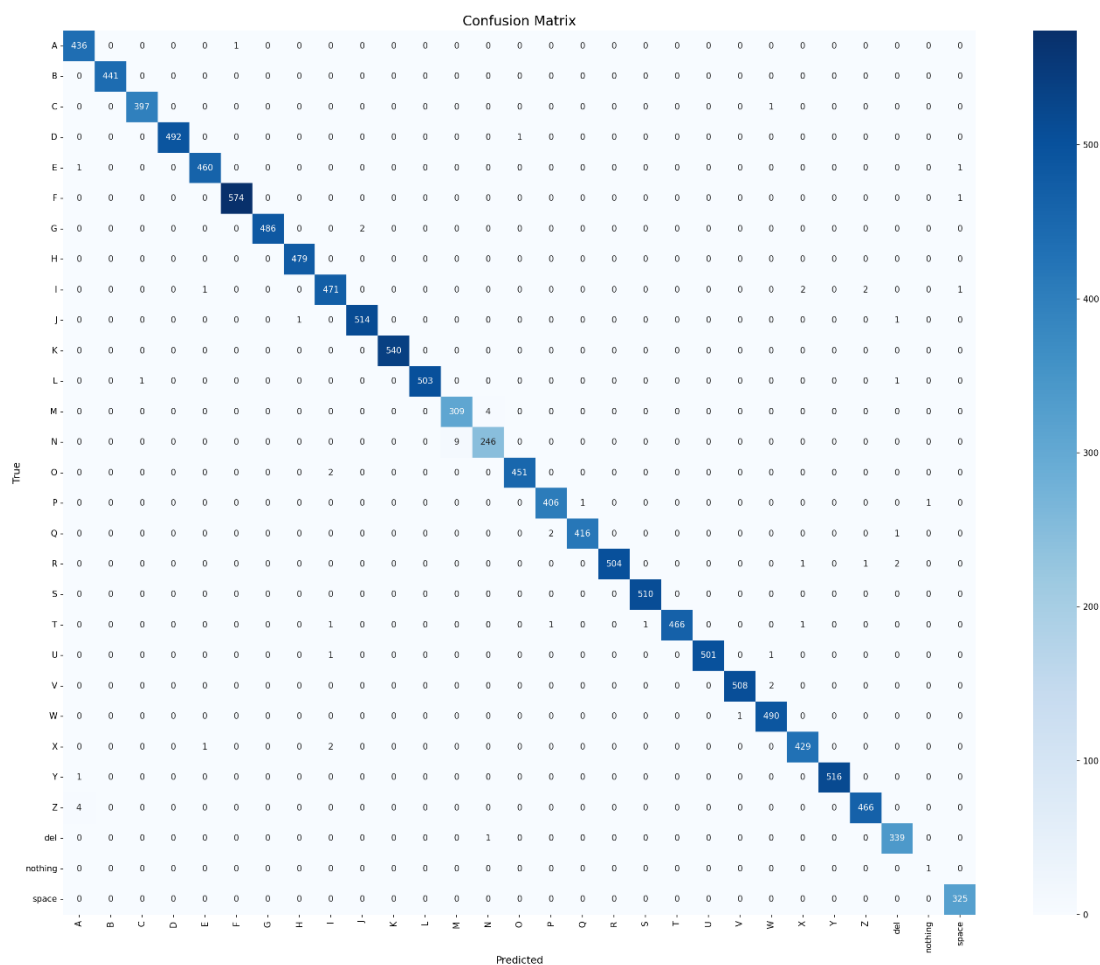
This confirms the model’s strong ability to correctly detect each ASL sign with very minimal misclassification.

Only a few gestures (e.g., M and N) show slightly lower scores due to high visual similarity in their hand shapes.

• **Confusion Matrix:**

The confusion matrix shows that the model makes very few mistakes across all sign categories. Most entries lie on the diagonal, confirming that the predicted class usually matches the true class.

Misclassifications — where present — occur between visually similar signs, which is expected due to close resemblance in hand-pose geometry.

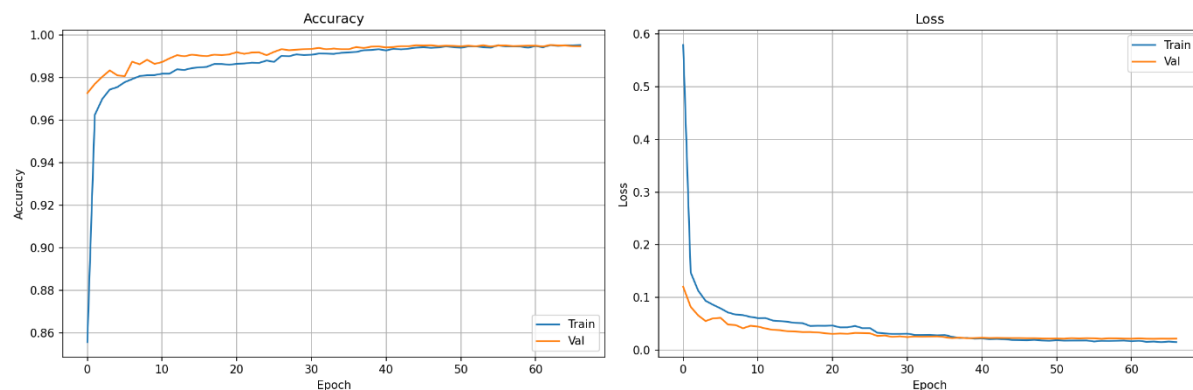


• Training Curves:

Training curves for accuracy and loss indicate:

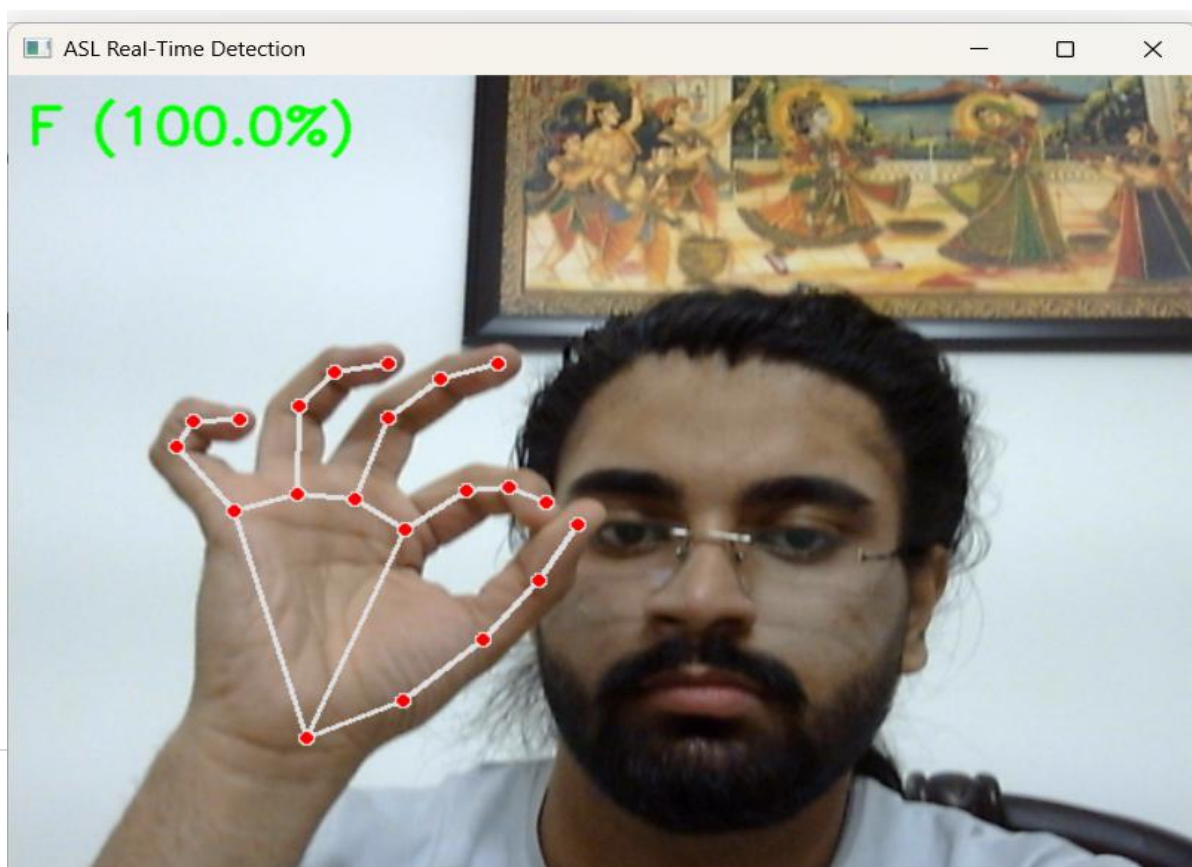
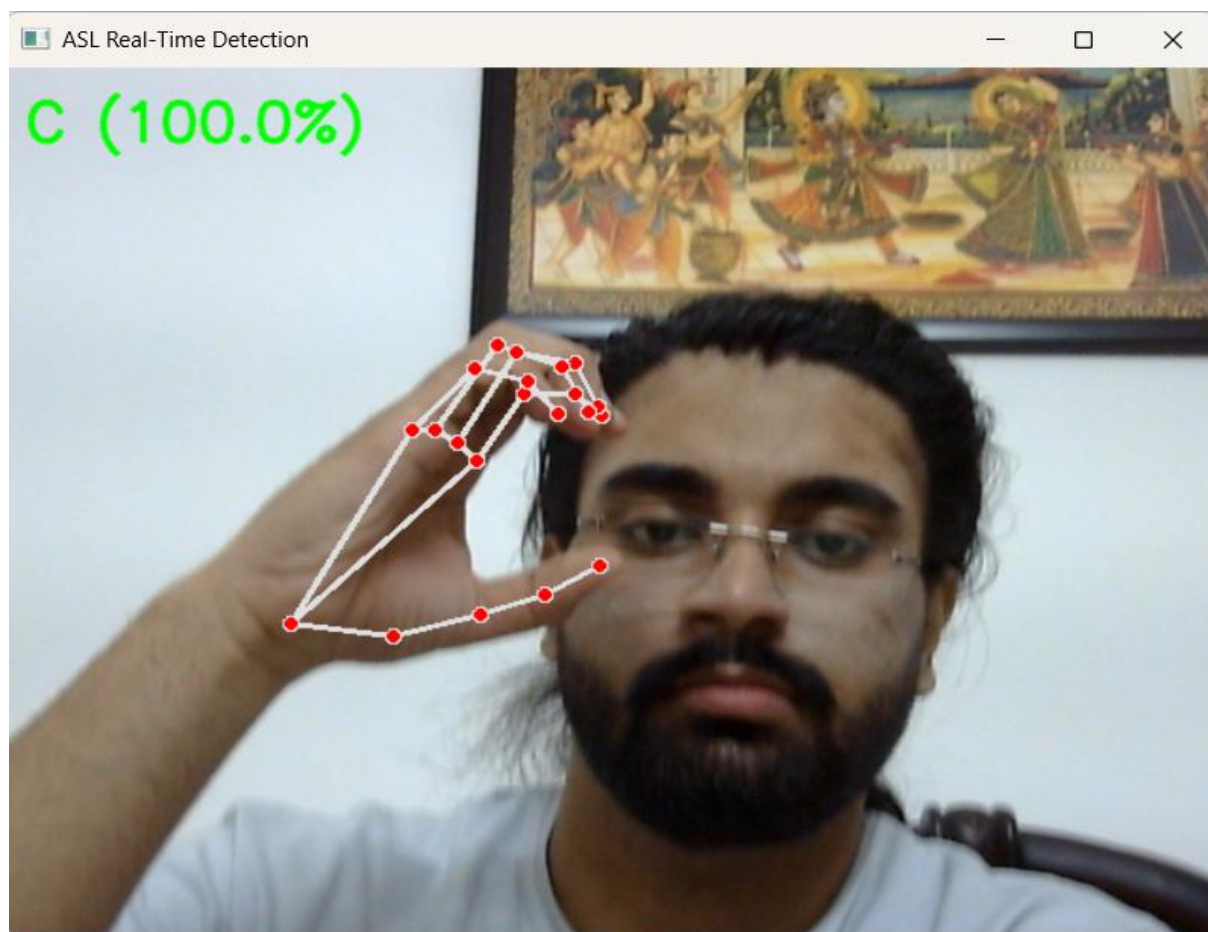
- Rapid improvement during early epochs
- Stable convergence
- Narrow gap between training and validation curves
- No significant overfitting

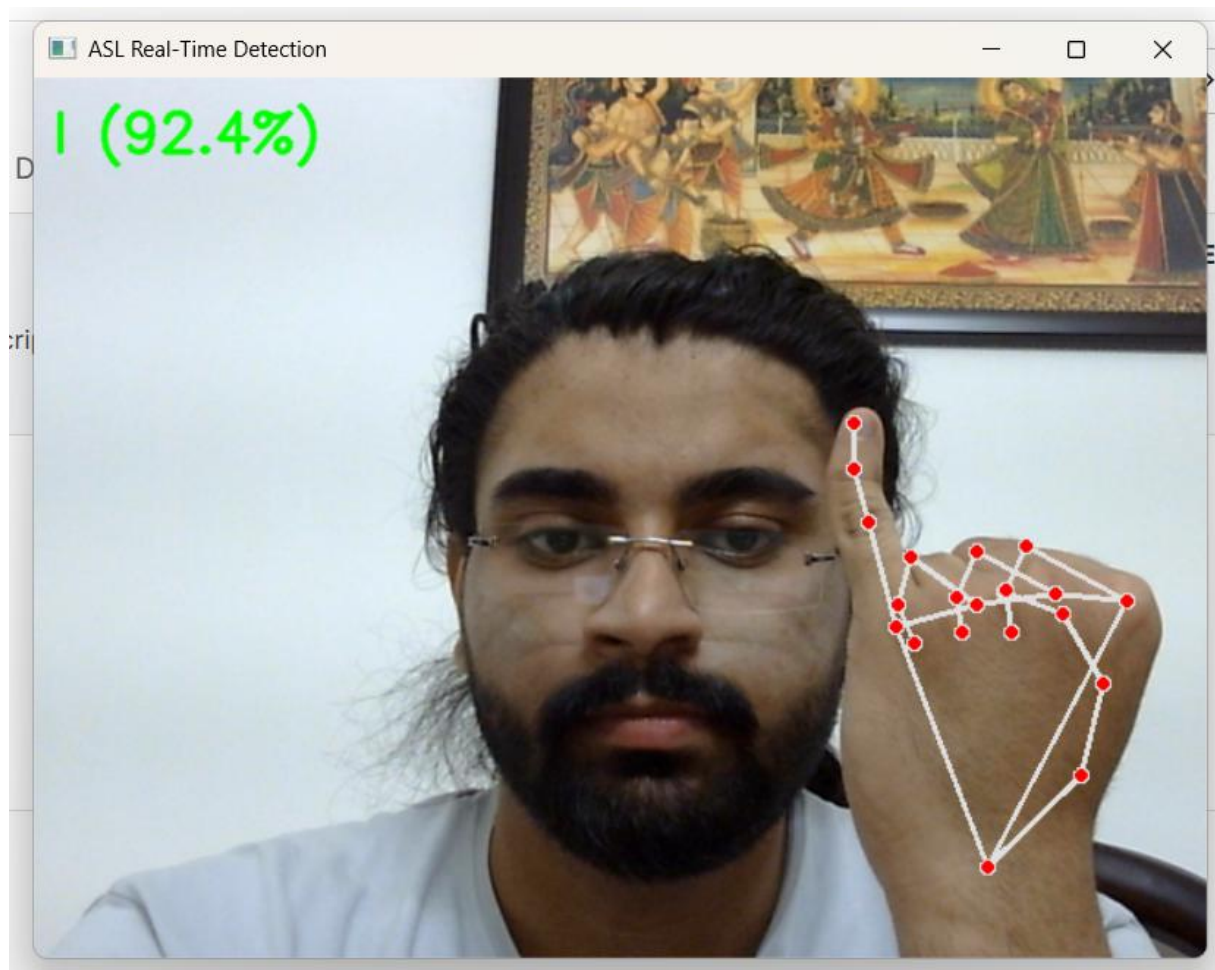
The model's learning behaviour remains consistent and reliable across the entire training process.



- **Real-Time Performance:**

In addition to offline evaluation, the model was integrated into a live webcam-based ASL recognition system using OpenCV and MediaPipe.





Conclusion:

This project successfully demonstrated how deep learning and computer vision can be used to recognise American Sign Language (ASL) gestures with high accuracy. By using the ASL Alphabet dataset from Kaggle and extracting 3D hand landmarks through MediaPipe, the system transformed raw images into structured numerical features suitable for machine-learning models. A Multilayer Perceptron (MLP) classifier was then trained on these features to learn the hand-pose patterns corresponding to different ASL letters.

The final model achieved outstanding performance, including:

- 99.53% test accuracy
- Extremely low loss values
- High precision, recall, and F1-scores across all 29 gesture classes
- A confusion matrix indicating very few misclassifications

These results clearly show that a landmark-based deep learning approach is highly effective for ASL alphabet recognition. The combination of MediaPipe's robust hand-tracking and the neural network's ability to learn gesture variations enables reliable and efficient gesture classification. Beyond training, the model was successfully integrated into a real-time detection pipeline using a webcam, allowing live sign recognition frame by frame. This demonstrates the practical potential of the project in real-world applications such as assistive communication tools, educational platforms, or gesture-controlled systems.

Overall, this project highlights how AI-driven techniques can make sign-language recognition more accessible, accurate, and scalable. It serves as a strong foundation for future work,

including full-word recognition, sentence-level interpretation, and deployment on mobile or embedded devices.

Appendix:

1. Training Code:

```
In [1]: print("Installing required packages...")
        print("=" * 60)
        !pip install -q mediapipe opencv-python-headless
        print("All packages installed successfully!\n")
```

```
In [2]: print(" Importing libraries...")
        print("=" * 60)
        import os
        import cv2
        import mediapipe as mp
        import numpy as np
        import pandas as pd
        import pickle
        from tqdm.notebook import tqdm
        import warnings
        warnings.filterwarnings('ignore')
        print("All libraries imported successfully!\n")
```

```
In [3]: print("Setting up configuration...")
        print("=" * 60)
        DATASET_PATH = "/kaggle/input/asl-alphabet/asl_alphabet_train/asl_alphabet_train"
        print(f"Dataset path: {DATASET_PATH}\n")
```

```
In [4]: print("Initializing MediaPipe Hand Detection...")
print("=" * 60)
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    static_image_mode=True,
    max_num_hands=1,
    min_detection_confidence=0.5
)
print("MediaPipe initialized successfully!\n")
```

```
In [5]: def extract_hand_landmarks(image_path):
    try:
        img = cv2.imread(image_path)
        if img is None:
            return None
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        results = hands.process(img_rgb)

        if results.multi_hand_landmarks:
            hand_landmarks = results.multi_hand_landmarks[0]
            wrist = hand_landmarks.landmark[0]
            flat_landmarks = []
            for landmark in hand_landmarks.landmark:
                flat_landmarks.append(landmark.x - wrist.x)
                flat_landmarks.append(landmark.y - wrist.y)
                flat_landmarks.append(landmark.z - wrist.z)
            return flat_landmarks
        return None
    except:
        return None
```

In [6]:

```
print(" Starting feature extraction...")
print("=" * 60)

landmark_data = []
stats = {'total_processed': 0, 'successful': 0, 'failed': 0}

class_folders = sorted([f for f in os.listdir(DATASET_PATH)
                        if os.path.isdir(os.path.join(DATASET_PATH, f))])

for label_folder in class_folders:
    folder_path = os.path.join(DATASET_PATH, label_folder)
    print(f"\n Processing class: {label_folder}")

    try:
        image_files = [f for f in os.listdir(folder_path)
                        if f.lower().endswith(('.jpg', '.png', '.jpeg'))]
        if not image_files:
            print(f" No images in {label_folder}")
            continue
    except Exception as e:
        print(f" Error: {e}")
        continue

    for image_name in tqdm(image_files, desc=f" {label_folder}", leave=False):
        image_path = os.path.join(folder_path, image_name)
        stats['total_processed'] += 1

        landmarks = extract_hand_landmarks(image_path)

        if landmarks is not None:
            landmark_data.append([label_folder] + landmarks)
            stats['successful'] += 1
        else:
            stats['failed'] += 1

    print(f" Processed {len(image_files)} images")

try:
    hands.close()
except:
```

```

print(f" Processed {len(image_files)} images")

try:
    hands.close()
except:
    pass

print("\n" + "=" * 60)
print(" EXTRACTION COMPLETE!")
print(f" Total: {stats['total_processed']}")
print(f" Success: {stats['successful']}")
print(f" Failed: {stats['failed']}")
print(f" Rate: {(stats['successful']/stats['total_processed']*100):.2f}%\n")

```

In [7]:

```

print("Creating dataset...")
print("=" * 60)

column_names = ['label']
for i in range(21):
    column_names.extend([f'x{i}', f'y{i}', f'z{i}'])

df = pd.DataFrame(landmark_data, columns=column_names)

print(f" Shape: {df.shape}")
print(f"Classes: {df['label'].nunique()}")

```

In [8]:

```

print("\n Class distribution:")
print("=" * 60)
class_counts = df['label'].value_counts().sort_index()
print(class_counts)

```

```

Class distribution:
=====
label
A          2187

```

In [9]:

```
print("\n Saving...")
print("=" * 60)
df.to_pickle("asl_landmarks.pkl")
print(" Saved: asl_landmarks.pkl")
```

Saving...

=====

Saved: asl_landmarks.pkl

In [12]:

```
print("Importing libraries...")
print("=" * 60)

import os
import numpy as np
import pandas as pd
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from tensorflow.keras.optimizers import Adam

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

print(f" TensorFlow version: {tf.__version__}")
print(f" GPU Available: {tf.config.list_physical_devices('GPU')}")
print(" All libraries imported successfully!\n")
```

Importing libraries...

=====

TensorFlow version: 2.18.0
GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU')]
All libraries imported successfully!

In [13]:

```
print(" Loading processed data...")
print("=" * 60)

df = pd.read_pickle("/kaggle/input/signlanguage/asl_landmarks.pkl")
print(f" Loaded!")
print(f" Shape: {df.shape}")
print(f" Classes: {df['label'].nunique()}\n")
```

Loading processed data...

=====

Loaded!

Shape: (63676, 64)

Classes: 29

In [14]:

```
print(" Preparing data...")
print("=" * 60)

X = df.drop('label', axis=1).values
y = df['label'].values

print(f" Features: {X.shape}")
print(f" Labels: {y.shape}")

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
num_classes = len(label_encoder.classes_)
print(f" Classes: {num_classes}\n")
```

Preparing data...

=====

Features: (63676, 63)

Labels: (63676,)

Classes: 29

In [15]:

```
print(" Splitting data...")
print("=" * 60)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

print(f" Train: {X_train.shape[0]}")
print(f" Test: {X_test.shape[0]}\n")
```

Splitting data...

=====

Train: 50940

Test: 12736

In [16]:

```
print(" Scaling features...")
print("=" * 60)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(" Features scaled!\n")
```

In [17]:

```
print(" Building model...")
print("=" * 60)

keras.backend.clear_session()

model = Sequential([
    Dense(256, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(128, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),

    Dense(64, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print(" Model built!")
model.summary()
```


In [18]:

```
print("\n Setting up callbacks...")
print("=" * 60)

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = ReduceLR0nPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=5,
    min_lr=1e-7,
    verbose=1
)

checkpoint = ModelCheckpoint(
    'best_asl_model.h5',
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

callbacks = [early_stopping, reduce_lr, checkpoint]
print(" Callbacks ready!\n")
```

In [19]:

```
print(" Training...")
print("=" * 60)

history = model.fit(
    X_train_scaled, y_train,
    validation_data=(X_test_scaled, y_test),
    epochs=100,
    batch_size=128,
    callbacks=callbacks,
    verbose=1
)

print("\n Training complete!\n")
```

In [20]:

```
print(" Evaluating...")
print("=" * 60)

model.load_weights('best_asl_model.h5')

y_pred = model.predict(X_test_scaled, verbose=0)
y_pred_classes = np.argmax(y_pred, axis=1)

test_accuracy = accuracy_score(y_test, y_pred_classes)
train_loss, train_accuracy = model.evaluate(X_train_scaled, y_train, verbose=0)
test_loss, _ = model.evaluate(X_test_scaled, y_test, verbose=0)

print(f" Results:")
print(f" Train Accuracy: {train_accuracy*100:.2f}%")
print(f" Test Accuracy: {test_accuracy*100:.2f}%")
print(f" Train Loss: {train_loss:.4f}")
print(f" Test Loss: {test_loss:.4f}\n")
```

In [21]:

```
print(" Classification Report:")
print("=" * 60)
report = classification_report(
    y_test, y_pred_classes,
    target_names=label_encoder.classes_,
    digits=3
)
print(report)
```

In [22]:

```
print("\n Confusion Matrix...")
print("=" * 60)

cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(20, 16))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)
plt.title('Confusion Matrix', fontsize=16)
plt.xlabel('Predicted', fontsize=12)
plt.ylabel('True', fontsize=12)
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()
```

In [23]:

```
print("\n Training curves...")
print("=" * 60)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].plot(history.history['accuracy'], label='Train')
axes[0].plot(history.history['val_accuracy'], label='Val')
axes[0].set_title('Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history.history['loss'], label='Train')
axes[1].plot(history.history['val_loss'], label='Val')
axes[1].set_title('Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('training_history.png', dpi=150)
plt.show()
```

In [24]:

```
print("\n Saving files...")
print("=" * 60)

model.save('asl_model.h5')
print(" asl_model.h5")

with open('label_encoder.pkl', 'wb') as f:
    pickle.dump(label_encoder, f)
print(" label_encoder.pkl")

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
print(" scaler.pkl")

print("\n" + "=" * 60)

print("=" * 60)
print("\n Download these 3 files:")
print(" • asl_model.h5")
print(" • label_encoder.pkl")
print(" • scaler.pkl")
```

2. Real-Time Code:

```
live.py × signnnn.ipynb

1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  import pickle
5  import tensorflow as tf
6
7
8  MODEL_PATH = "asl_model.h5"
9  SCALER_PATH = "scaler.pkl"
10 ENCODER_PATH = "label_encoder.pkl"
11
12 print(" Loading model and preprocessors...")
13 model = tf.keras.models.load_model(MODEL_PATH)
14
15 with open(SCALER_PATH, "rb") as f:
16     scaler = pickle.load(f)
17
18 with open(ENCODER_PATH, "rb") as f:
19     label_encoder = pickle.load(f)
20
21 print(" Model, scaler, and label encoder loaded!")
22
23
24 mp_hands = mp.solutions.hands
25 mp_drawing = mp.solutions.drawing_utils
26
27 hands = mp_hands.Hands(
28     static_image_mode=False,
29     max_num_hands=1,
30     min_detection_confidence=0.5,
31     min_tracking_confidence=0.5
32 )
```



```
31     min_tracking_confidence=0.5
32 )
33
34 1 usage
35
36 def extract_hand_landmarks_from_frame(frame_bgr):
37     """
38     Input: BGR frame from OpenCV
39     Output: list of 63 features (21 landmarks * 3) or None
40     """
41
42     img_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
43     results = hands.process(img_rgb)
44
45     if results.multi_hand_landmarks:
46         hand_landmarks = results.multi_hand_landmarks[0]
47         wrist = hand_landmarks.landmark[0]
48
49         flat_landmarks = []
50         for lm in hand_landmarks.landmark:
51             flat_landmarks.append(lm.x - wrist.x)
52             flat_landmarks.append(lm.y - wrist.y)
53             flat_landmarks.append(lm.z - wrist.z)
54
55         return flat_landmarks, hand_landmarks
56     return None, None
57
58 cap = cv2.VideoCapture(0)
59
60 if not cap.isOpened():
61     print(" Could not open webcam.")
62     exit()
```

```
60     exit()
61
62     print(" Starting real-time ASL detection... (Press 'q' to quit)")
63
64     while True:
65         ret, frame = cap.read()
66         if not ret:
67             print(" Failed to grab frame.")
68             break
69
70
71         frame = cv2.flip(frame, flipCode: 1)
72
73
74         features, hand_landmarks = extract_hand_landmarks_from_frame(frame)
75
76         label_text = "No hand detected"
77         color = (0, 0, 255)
78
79         if features is not None:
80
81             X = np.array(features).reshape(1, -1)
82             X_scaled = scaler.transform(X)
83
84
85             preds = model.predict(X_scaled, verbose=0)
86             class_id = np.argmax(preds, axis=1)[0]
87             class_label = label_encoder.inverse_transform([class_id])[0]
88             confidence = float(np.max(preds))
89
90             label_text = f"{class_label} ({confidence*100:.1f}%)"
91             color = (0, 255, 0)
```


live.py × signnnn.ipynb

```
90     label_text = f"{class_label} ({confidence*100:.1f}%)"
91     color = (0, 255, 0)
92
93
94     mp_drawing.draw_landmarks(
95         frame,
96         hand_landmarks,
97         mp_hands.HAND_CONNECTIONS
98     )
99
100
101     cv2.putText(
102         frame,
103         label_text,
104         org: (10, 40),
105         cv2.FONT_HERSHEY_SIMPLEX,
106         fontScale: 1.0,
107         color,
108         thickness: 2,
109         cv2.LINE_AA
110     )
111
112     cv2.imshow( winname: "ASL Real-Time Detection", frame)
113
114
115     if cv2.waitKey(1) & 0xFF == ord('q'):
116         break
117
118
119     cap.release()
120     hands.close()
121     cv2.destroyAllWindows()
```

```
117
118
119     cap.release()
120     hands.close()
121     cv2.destroyAllWindows()
122     print(" Exited.")
123
```